Module 1

Introduction to Reinforcement Learning

The primary goal of the presented modules on reinforcement learning (RL) design projects is enhancement of existing curricula and the development of new curricula on RL for undergraduate and graduate students in engineering. The designed projects will also benefit undergraduate and graduate students and instructors, in computer science, sciences, economy, business, and operational research. The RL projects, exercises, and homework problems presented in these modules come from all areas of engineering with emphasis on real physical systems such as aircraft, spacecraft, drones, robots, vehicles, green energy systems (such as fuel cells), bioengineering, and in general diverse dynamic systems encountered in industry.

In the past ten to fifteen years RL has become an important engineering teaching and research area. Several universities have started offering graduate and undergraduate courses on RL in their engineering curricula. The projects are implemented in MATLAB so that practical working knowledge of MATLAB and Simulink is expected from the readers, particularly students, and practicing engineers. The modules provide variety of RL design projects with their solutions using MATLAB's live scripts and functions, and Simulink.

Each designed project has an independent and self contained very brief presentation of analytical results on the subject under consideration, and with detailed RL design project formulations. We encourage students and instructors to master MATLAB and relevant MATLAB toolboxes by visiting the MathWorks website and/or to take the MathWorks abbreviated courses: "MATLAB Onramp", "Simulink Onramp" and "Reinforcement Learning Onramp". In Module 2, we will provide a short Simulink Manual and a brief summary of the main MATLAB functions from the Control Systems Toolbox relevant to the RL projects designed.

Our plan is to develop a set of MATLAB, Simulink, and Control System Toolbox oriented *design projects, exercises, and homework problems* that will make RL easily understandable and simpler for using by a broad educational community, first of all students in all engineering fields and some areas in sciences, operational research, economy, and business. The developed material is based on the engineering approach to RL from the feedback control systems perspective, which in fact represents iterative, adaptive, and optimal feedback control of dynamic systems. The central theme evolves around the *approximate dynamic programming* techniques. The controls and systems approach is used to generate learning policies (optimal feedback strategies, optimal feedback controls).

Our approach is based on optimal feedback control systems and Bellman's optimality principle. Sutton and Barto in their well-known textbook [2] wrote: "We formalize the problem of reinforcement learning using ideas from dynamic systems theory,..." and "One of the most pressing areas for future reinforcement learning research is to adopt and extend methods

developed in control engineering with the goal of making it acceptably safe to fully embedded reinforcement learning agents into physical environments".

RL means learning from interactions. This expectation is due to specifics of RL that it is much closer the way humans think and operate than neural networks based supervised and unsupervised learning methods, as pointed out by Sutton and Barto [2]: "Of all the forms of machine learning, reinforcement learning is the closest to the kind of learning that humans and other animals do,..." Moreover, the same authors in the same book wrote: "Reinforcement Learning has gradually become one of the most active research areas in machine learning, artificial intelligence, and neural network research."

The designed projects are mostly based on iterative linear *optimal feedback control of dynamic systems*. The knowledge of feedback systems is important, but not essential to successfully master all design steps and methodology of presented projects and fully understand their solutions. *Solid mathematical background is required*, such as knowledge of linear algebra, optimization, differential and difference equations, some knowledge of dynamic systems and controls. The projects designed are introductory in nature, and advanced senior undergraduate students in engineering with strong mathematical background will be able to master them successfully.

The approach taken is deterministic, which is specific to the engineering approach to RL. Some schools and researchers, particularly in computer science, consider RL using stochastic problem formulations based on Markov decision processes and Monte Carlo methods. RL has been developing over many years using both deterministic and stochastic formulations in diverse fields, such as: computer science, mathematics, statistics, engineering, operational research, economics, business, finance, phycology (cognitive sciences), neuroscience, and other scientific disciplines.

Several *overview papers* have provided very comprehensive surveys of the development of RL within the past fifteen years, with emphasis on engineering applications to real physical systems, [3]-[6]. In addition, there are at least two dozen books on RL written within computer science and a dozen of books on control engineering approach to RL, see for example, [7]-[11].

1.1 Brief History, Achievements, and Relation to Optimal Control

The first solution to the dynamic optimization problem was derived by Richard Bellman in the 1950s using the technique developed by himself that he called dynamic programming. The results of Bellman were published in his famous 1957 book titled *Dynamic Programming* [1]. The dynamic programming methodology can be used to solve the general optimal control problem in both deterministic and stochastic frameworks, and in both continuous- and discrete-time domains.

RL has gained its maturity within computer science in the past thirty or so years, and has become an important research and educational topic in engineering in the past fifteen years, with more engineering schools offering RL undergraduate and graduate classes every year, and with more and more engineering faculty being involved in research on RL.

The engineering approach to RL is based on approximate dynamic programming (ADP), also known as adaptive dynamic programming. It has been known since the 1960s that dynamic programming solves dynamic optimization (optimal control) problems of continuous- and discrete-time. However, dynamic programming suffered from analytical and computational

difficulties so that it took long time, until the beginning of this century, when ADP was discovered, and when ADP has become a powerful RL technique.

RL in general represents a computational method for learning from feedback received information through interactions with environment that is represented by a dynamic system. RL can be model-based (when a mathematical model of the dynamic system exists) and model-free when no such a model is available. The approach taken in this book to RL will be either *model-based* or *partially model-based* or completely *model-free*, with applications to engineering systems whose dynamics can be described either by differential or difference equations. *The central theme in all presented methods will be discovering the built-in mechanism (algorithm, recursive program, or a set of repeated actions) that iteratively improves control strategies through RL schemes such that RL strategies converge to the optimal solutions (optimal control, optimal system state trajectories, optimal system performance)* for considered engineering dynamic systems.

RL researchers in computer science consider *RL as an approximate optimal control technique*. The title of the paper by computer science researchers, [13], is "Reinforcement Learning is Direct Adaptive Optimal Control." In addition to *approximate dynamic programming,* RL is also known under several different names: *adaptive dynamic programming* or *heuristic programming* or *neuro-dynamic programming* or *iterative dynamic programming* or *incremental dynamic programming* or *forward dynamic programming*.

The control system approach to RL culminated in the past fifteen years in the works of Frank Lewis and his coworkers at University of Texas at Arlington, who published several books and a large number of journal papers on RL, [3]-[5], [10]-[11].

1.2 Computer Science Basic RL Techniques

In the 1980s, the temporal-difference methods [13]-[14] were developed, followed by the Q-learning methods [15]-[16]. The *temporal difference (TD) learning* method that originated in the dissertation of Sutton [13] and the *Q-learning* method that originated in the dissertation of Watkins [15] revolutionized the RL techniques within computer science. These days RL is considered *as* an approximate optimal control technique (ADP, [17]). The temporal difference learning and the Q-learning techniques can be represented in their basic forms as follows.

Temporal Difference (TD) *Learning* [13]-[14]: A *state value function* $V(s)$, where s are state variables, is optimized using the TD algorithm, whose original formulation is given by [1]:

$$V(s_k) \leftarrow V(s_k) + \gamma \left\lfloor V(s_{k+1}) - V(s_k) \right\rfloor = (1-\gamma)V(s_k) + \gamma V(s_{k+1}) \tag{1.1}$$

where $V(s_k)$ is the value (or estimate) function at present state s_k. $V(s_{k+1})$ is the value (or estimate) function at the next state s_{k+1}. $V(s_{k+1}) - V(s_k)$ is *the temporal difference*, k is the present time, $k+1$ is the next time instant. γ is the step size, often called the *learning rate*.

Q-Learning [15]-[16] includes the action (control) variable a so that the *action-value function denoted by* $Q(s,a)$ is optimized using the algorithm:

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \gamma \left\lfloor Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k) \right\rfloor \tag{1.2}$$

where, in addition to quantities defined in (1.1), a_k is the action (control) signal at time k and a_{k+1} is the action (control) signal at time $k+1$.

The original TD-learning method (1.1) and the Q-learning method (1.2) are one-step iterative schemes, both based on a simple strategy:

$$\text{NEW ESTIMATE} \leftarrow \text{OLD ESTIMATE} + \gamma \times \left[\text{ERROR ESTIMATE}\right] \tag{1.3}$$

Both TD-learning and Q-learning have several variants, especially the ones that incorporate expected values and discount factors. MATLAB (and many other publications) call the following variant (using MATLAB notation) of Q-learning as the "*Bellman Equation*"

$$\text{new}\, Q(s,a) = Q(s,a) + \alpha \left[R(s,a) + \gamma \max Q'(s',a') - Q(s,a) \right] \tag{1.4}$$

In (1.4), $\text{new}\, Q(s,a)$ is the new estimate of the state-action value, $Q(s,a)$ is the previous estimate of the value, α is the step size also called the learning rate, $R(s,a)$ is the reward for taking action a at state s, γ *is the discount factor for future awards*, $\max Q'(s',a')$ is the maximum expected value from state s'. The term $R(s,a) + \gamma \max Q'(s',a')$ represents the best new estimate of the value. It corresponds to $Q(s_{k+1}, a_{k+1})$ in (1.2), which indicates the equivalence of (1.2) and (1.4). The Bellman equation (1.4) has more practical structure than the original Q-learning formula (1.2) due to introduction of the reward function and the discount factor for future awards. Several variants of (1.4) can be found in the RL literature with some of them presented in [2].

1.3 General Reinforcement Learning Block Diagram and MATLAB Live Editor

A general RL block diagram is essentially an expanded feedback control system block diagram as presented in Fig. 1.1, which in addition to the classical feedback loop from the state variables (lower feedback loop) has another (upper) feedback loop that provides information about the performance (cost, reward) at all times using information about the state variables.

MATLAB's Live Editor environment is very convenient for implementation of the RL mechanism. In Module 2, we will briefly review features of the MATLAB Live Editor and corresponding live scripts and live functions, as well provide a sufficient introduction to Simulink. In this section, using the block diagram from Figure 1.1 we indicate how particular blocks of the RL mechanism can be implemented using MATLAB's live scripts, live functions, and Simulink.

It can be seen from Figure 1.1 that *the control agent can be implemented using a MATLAB live script (program). The state of the system upon given control input (action), obtained via a feedback channel measurement, can be represented by a MATLAB live function. The reward (cost, performance) estimation via the upper feedback loop can be obtained using a MATLAB live function. The environment (engineering system subjected to RL) can be modeled using Simulink (or in simpler cases using a live function)*. Even more, in some simpler RL problems, live functions for particular blocks in Figure 3.1 can be replaced by (ordinary, common) MATLAB functions.

The MATLAB triad: live script, live function, Simulink will play essential roles in the developed RL design projects for engineering systems to be presented in the follow-up modules that

correspond to different formulations: discrete, continuous, single or multiple agents, exactly or partially known environment (either the system mathematical model or the actual physical system) or even completely unknown environment that only reveals either its states or the approximate performance values via feedback signals. As a matter of fact, the *MATLAB live scripts interactive capabilities in conjunction with Simulink modeling and simulation capabilities can create an excellent, software oriented testbed for verification of reinforcement learning algorithms for engineering systems*.

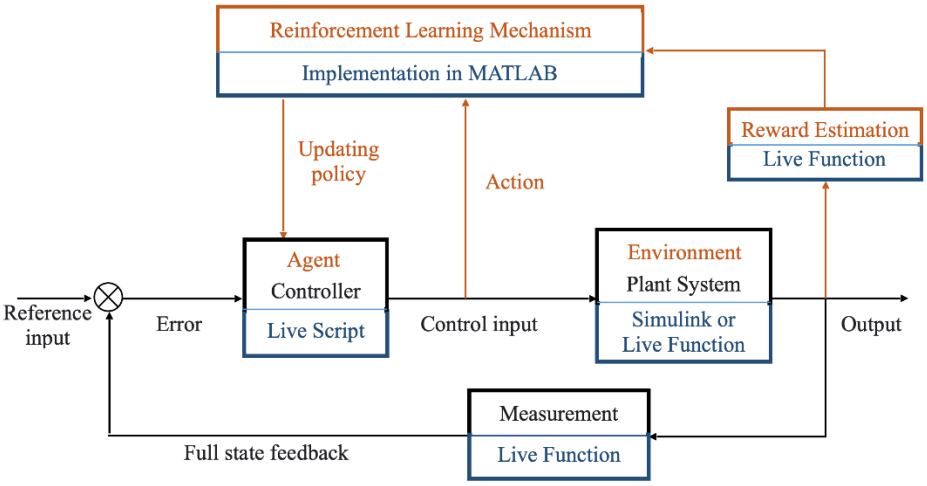


Figure 1.1: MATLAB implementation of the reinforcement learning mechanism

It is the opinion of the authors of these modules that students, faculty, RL researchers, and practicing engineers do not need a special *Reinforcement Learning Manual for Engineering Dynamic Systems*. They just need to master well the MATLAB live editor with its live scripts and live functions, and Simulink's tremendous dynamic system modeling and simulation capabilities in order to be able to implement almost any RL algorithm dealing with real world engineering dynamic systems.

1.4 Organization of Modules

The first two modules are introductory, presenting an introduction to RL and introduction to Simulink and MATLAB's basic RL functions and its Live Editor. The remaining modules are organized into various types of design projects, each presented as an independent entity. In addition to the elementary RL theory on a particular RL topic, basic MATLAB codes and standard demos, are provided. The modules are organized as follows:

PART I: INTRODUCTION

Module 1 : Introduction to RL with comparison of computer science and engineering RL approaches.

Module 2: Short Simulink manual and some functions form Control System Toolbox important for RL, including an introduction to MATLAB live scripts and live functions.

PART II: MODEL BASED REINFORCERMENT LEARNING

Module 3: Introduction to dynamic programming in discrete-time. The Hamiltonian and the Q-function. Discrete-time linear-quadratic (LQ) optimal control and the Hewer algorithm.

Module 4: Introduction to dynamic programming in continuous time. LQ optimal control and the algorithm of Vaisbord-Millstein-Kleinman. RL for affine nonlinear optimal control systems.

Module 5: Successive approximations in continuous- and discrete-time domains. Continuous- and discrete-time policy iterations and value iterations.

Module 6: RL via policy iterations for zero-sum differential games.

Module 7: RL for Nash differential games.

PART III: MODEL FREE REINFORCERMENT LEARNING

Module 8: RL for partially model free dynamic systems.

Module 9: RL for completely model free reinforcement learning.

Module 10: Model free RL for zero-sum differential games.

Module 11: Model free RL for Nash differential games.

1.5 References

[1] Bellman, R., *Dynamic Programming*, Princeton University Press, Princeton NJ, 1957. Republished by Dover Publications Inc., Mineola NY, 2003.

[2] Sutton, R. and A. Barto, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, MA, 2020.

[3] Lewis, F. and D. Vrabie, "Reinforcement learning and adaptive dynamic programming for feedback control," *IEEE Circuits and Systems Magazine*, 32-50, Third Quarter, 2009.

[4] Lewis, F., D. Vrabie, and K. Vamvoudakis, "Reinforcement learning and feedback control," *IEEE Control Systems Magazine*, 76-105, 2012.

[5] Kiumarsi, B., K. Vamvoudakis, H. Modares, and F.L. Lewis, "Optimal and autonomous control using reinforcement learning: A survey," *IEEE Transactions Neural Networks and Learning Systems*, Vol. 29, 2042-2061, 2018.

[6] Wang, X., S. Wang, X. Liang, D Zhao, J. Huang, and X. Xu, "Deep reinforcement learning: A Survey," *IEEE Transactions on Neural Networks and Learning Systems*, Vol. 35, 5064-5078, 2024.

[7] Bertsekas, D. and J. Tsitsiklis, *Neuro-Dynamic Programming*, Athena Scientific, Nahua, NH, 1996.

[8] Bertsekas, D., *Reinforcement Learning and Optimal Control*, Athena Scientific, Belmont, MA, 2019.

[9] Bertsekas, D., *A Course in Reinforcement Learning*, Athena Scientific, Belmont, MA, 2023.

[10] Vrabie, D., K. Vamvoudakis, and F. Lewis, *Optimal Adaptive Control and Differential Games by Reinforcement Learning Principles*, IET, London, United Kingdom, 2013.

[11] Lian, B., W. Xue, F. Lewis, H. Modares, and B. Kiumarsi, *Integral and Inverse Reinforcement Learning for Optimal Control Systems and Games*, Springer, 2024.

[12] Sutton, R., A. Barto, and R. Williams, "Reinforcement learning is direct adaptive optimal control," *IEEE Control Systems Magazine*, Vol. 12, 19-22, 1992.

[13] Sutton, R., *Temporal Credit Assignment in Reinforcement Learning*, Ph. D. Dissertation, University of Massachusetts, Amherst, MA, 1984.

[14] Sutton, R., "Learning to predict by the methods of temporal differences," *Machine Learning*, Vol. 3, 9-44, 1988.

[15] Watkins, C. *Learning from Delayed Rewards*, Ph. D. Dissertation, University of Cambridge, MA, 1989.

[16] Watkins, C. and P. Dayan, "Q-Learning," *Machine Learning*, Vol. 8, 279-292, 1992.

[17] Murray, J., C. Cox, G. Lendaris, and R. Sacks, "Adaptive dynamic programming," *IEEE Transactions on Systems, Man, and Cybernetics*, Vol. 32, 140-153, 2002.